PRD — SIG Predictions Cup Trading System

Oct 2, 2026 · @Arnav

Overview and goals

We are building a real-time trading system for the SIG Predictions Cup 2026 (US midterm election markets, Oct 1 – Nov 4, 2026) aimed at a strong final rank and Smart Score through disciplined, risk-adjusted trading, concentrated on election night. It doubles as a resume project for quant, SWE and AI roles.

Decision (Oct 2): we do not chase top 3. Every market opened at 50c, and the leaders turned that into gains of 350k–625k SUSQies on day one; catching them would need concentrated tail bets that most likely end far down the table. Time budget is 1–2 hours a day, with exams Oct 17–26.

In plain terms: the system watches prices on SIG's platform, compares them with real-money prices on Polymarket and Kalshi, trades when SIG's price is wrong, protects itself with strict risk limits, and shows everything on a live dashboard.

Goals, in priority order

1. Strong final rank with low drawdown (top 100 of ~3,900 entrants).
2. Strong Smart Score, SIG's risk-adjusted rating from P&L, ROI, win rate and volatility, which feeds its recruiting.
3. Never lose SUSQies to a bug: no duplicate orders, no accidental position flips, no runaway trading.
4. Measured evidence for the resume: how far SIG lags Polymarket, election-night reaction speed, calibration.

Success metrics

Metric	Target
Final rank	Top 100 (stretch: top 25)
Maximum drawdown from peak balance	Under 10%
Order errors causing losses	0
Logger uptime, Oct 16 – Nov 4	⩾ 99%
Election-night reaction time to a Polymarket move	Under 5 seconds

Metric	Target
Repricing-lag study	Distribution of SIG's lag behind Polymarket, published in the post-mortem

Competition constraints

Every design decision below follows from these rules and API limits; breaking one costs either SUSQies or eligibility.

Constraint	Detail	Design consequence
Trading window	Oct 1, 9:30pm IST → Nov 4, 10:30pm IST (12pm ET both ends)	Election night falls in IST daytime, Nov 4, 4:30am–11:30am poll closings
Scoring	Balance after every market resolves	Idle cash at cutoff earns nothing; positions settle after lock
Payout	Only top 3 paid	Leaders gained 4–7x on opening day; we optimise risk-adjusted return and Smart Score instead
Rate limit	100 reads + 30 writes per minute, per account	WebSockets for data; batch orders (50 per write); every key shares this budget, so dev tools and agents use the mock exchange, never the live API
Tick size	0.005, prices 0.005–0.995, YES-denominated	Round every price; NO cost = 1 − price
Sell semantics	Selling more than held becomes buying the other side	Sells only close held positions; a short view is taken by buying NO
Idempotency	Every order needs an idempotencyKey	Retry 502/503 with the same key only
ALL collateral	Guaranteed payout floors can fund new fills	Verify with a tiny test trade before relying on it
One account, individual	Multi-accounts or SUSQie transfers = disqualification	No teammates trading for us
MNPI	No trading on non-public info	Public data only

Constraint	Detail	Design consequence
Data ownership	SIG owns competition data and derived analytics	Publish code, never raw dumps; repo private until after Nov 4
Resolution rules	Not returned by the API	Copy each market's Info tab into the repo by hand

Market facts from the Oct 2 dump

- 237 YES/NO markets across 117 races: Senate, Governor, about 40 competitive House seats, and House and Senate control. Each race has a Democratic and a Republican market; Montana Senate, Nebraska Senate and Rhode Island Governor also have an Independent one
- No composite, multi-outcome or linked markets, so the engine's constraint alerts and ALL collateral never apply
- Median spread is one tick (0.5c), with about 5,600 SUSQies of depth within 2c of mid; SIG's market maker visibly quotes 144 markets
- SIG prices track Polymarket and Kalshi closely (Senate control: SIG 63.5% D vs 63% averaged); Democratic/Republican pair gaps are 0.5–1c on a few hundred shares
- 752 active traders; the top 8 are up 350k–625k SUSQies after buying at the 50c opening prices
- Each contract carries a race ID, stage ID and resolution type "Party Winner"; the exact resolution source is still to confirm from the Info tabs

Scope and priorities

October builds a data logger and nothing that trades; the trading system is built after exams for election night. If time runs short we cut from the bottom of this table, never the top.

Priority	Component	Done by	Why it matters
Must	SIG API client (REST, WebSocket, rate limiter)	Oct 7	Everything else depends on it
Must	Logger: SIG, Polymarket and Kalshi quotes into SQLite, with the 117-race mapping	Oct 11	Data for the lag study and election night
Must	Fair value v1 and the `mispricings` command for manual trades	Oct 13	Your October trading tool

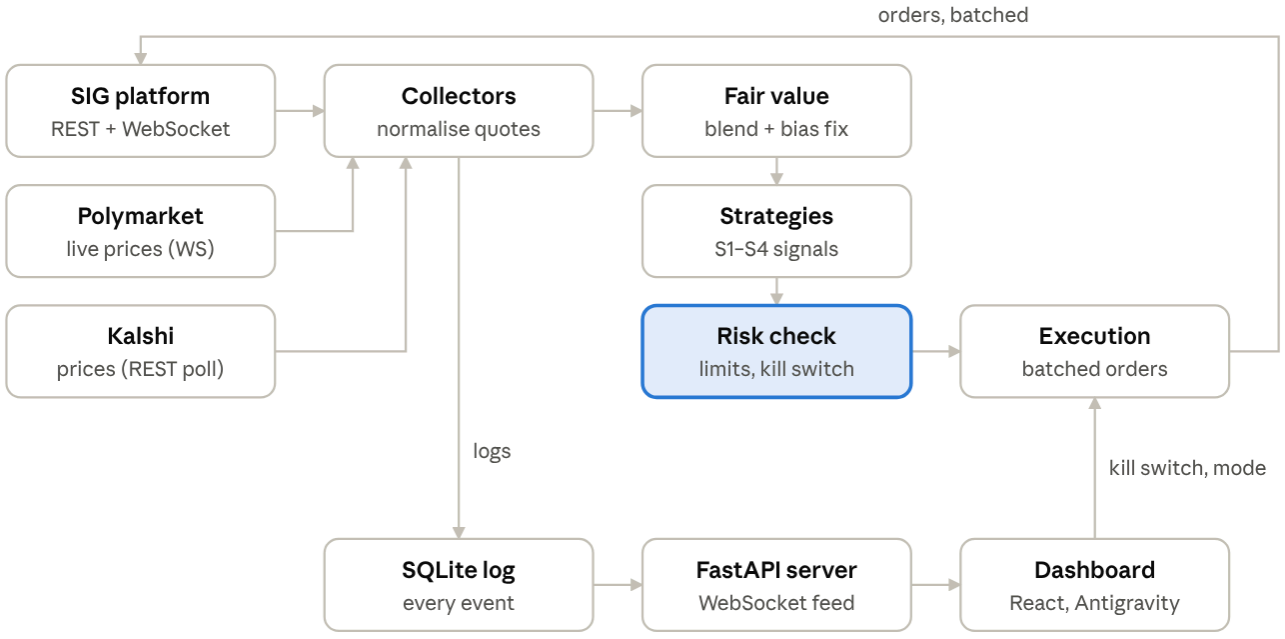
Priority	Component	Done by	Why it matters
Must	Logger on the cloud server, with a Telegram health alert	Oct 16	Runs untouched through exams
Must	Mock exchange	Oct 28 (drafted on a branch during exams)	Tests orders without risking SUSQies
Must	Execution, risk manager and kill switch	Oct 30	Places orders safely
Must	S4 election-night strategy	Oct 31	Where the rank is earned
Must	Terminal dashboard: F1 Monitor, F5 Election board, F6 Health, KILL	Oct 31 (shell drafted during exams)	Watch and stop the bot
Must	Rehearsal and runbook	Nov 2	Proves the system before it counts
Should	Lag-study report: how long SIG takes to follow Polymarket	Oct 28	Sizes the election-night edge; resume metric
Should	v0 scenario caps from each race's lean	Oct 30	Limits exposure to a polling miss
Should	Election-results recorder	Nov 2	Data for the November vote-count model
Nice	Dashboard F2 Market, F3 Risk, F4 Blotter, F7 Leaderboard	If time	Nicer to operate
Nice	Bayesian vote-count model, built on the recorded 2026 results	November	AI/ML resume piece, no trading risk

Cut on Oct 2: constraint arbitrage (pair gaps are 0.5–1c on a few hundred shares), passive quoting (SIG's market maker already quotes one-tick spreads), automated October trading, the full correlated simulator, the LLM market matcher and news triage. Always out of scope: reinforcement learning, deep-learning price prediction, and any LLM that places trades.

System architecture

One Python engine runs the whole loop on a cloud server; the dashboard only watches it and can stop it.

Every order passes one risk gate before reaching SIG



system architecture · data in, orders out, everything logged

Prices flow in from three venues, become one fair value per contract, turn into signals, and reach SIG only after the risk check. Every step writes to the SQLite log, which the FastAPI server streams to the dashboard.

Shared contracts

The engine (Claude Code) and dashboard (Antigravity) never share code; they share only these schemas, defined first in `shared/schemas.py` (Pydantic) and exported as JSON Schema for the dashboard.

Core data model

Object	Key fields	Notes
`Market`	sig_market_id, title, exchanges[], is_composite, resolution_text, settlement_date, lean (D / R / none)	resolution_text copied from the Info tab

Object	Key fields	Notes
`MarketMapping`	sig_exchange_id, polymarket_token_id, kalshi_ticker, match_confidence, mismatch_notes	Hand-built first; LLM matcher later
`Quote`	source (sig/polymarket/kalshi), exchange_id, bid, ask, mid, ts	YES-denominated; stored as integer ticks (price × 200) to avoid float errors
`FairValue`	exchange_id, value, inputs{}, model_version, ts	inputs records each source's weight
`Signal`	exchange_id, strategy, side, edge_cents, size, reason	reason is human-readable, shown on dashboard
`OrderIntent`	idempotency_key, exchange_id, side, action, qty, price	Passes risk checks before sending
`Fill`, `Position`, `RiskState`	Mirrors API Fill/Position; RiskState holds limits and kill-switch flag	

Engine → dashboard messages (WebSocket, JSON, one envelope)

```
{"type": "quote_update | fair_value | signal | order | fill | position | risk
| log | health",
 "ts": "2026-10-08T14:02:11Z",
 "data": { }}
```

Dashboard → engine commands (REST, password-protected): `POST /kill` (cancel all + stop), `POST /resume`, `POST /mode` (manual / election), and POST /override (set one contract's fair value by hand, expiring after 15 minutes). Nothing else: the dashboard cannot place trades, and overrides still pass the risk check.

Storage: SQLite (WAL mode) for October, every row timestamped; one table per object above. Parquet export nightly for analysis.

Strategies

October is for measurement and occasional manual trades, because SIG's market maker already prices off Polymarket and Kalshi; election night is where the system earns its rank.

Strategy	When	How it works	Entry rule (initial)
Manual trades	Oct 13 – Nov 2	You trade by hand when `mispricings` shows SIG at least 3c away from both Polymarket and Kalshi	Edge ⩾ 3c; ⩽ 2,000 SUSQies per trade; limit orders only
Lag study	Oct 11 – Nov 4	The logger measures how long SIG takes to follow each Polymarket move	No trading
S4 Election night	Nov 4, 4:30am – 10:30pm IST	Polymarket's live price is fair value, checked against Kalshi; the bot buys where SIG lags, spread across many races	Edge ⩾ 3c; fair value under 5 seconds old; scenario caps

Fair value v1: the mid of Polymarket and Kalshi, weighted by book depth; a source is ignored when its spread is wider than 4c or its quote is stale. For contracts under 10c or over 90c, the estimate is pushed slightly away from 50%, because real-money markets overprice long shots (a 5c contract is usually worth a little under 5c). Contracts with a resolution mismatch get no fair value until reviewed.

Capital plan: spread exposure across many races, never more than 10% of balance in one race, and cap net exposure to a uniform polling miss either way. No concentrated tail baskets. Enter election night with at least half the balance in cash; after 9:30pm IST on Nov 4, deploy remaining cash only where the edge is positive.

Risk management

Every order passes the same pre-trade checks in one function, `risk.check(intent)`, and any failure blocks the order and logs why.

Pre-trade checks (all modes)

- Price on the 0.005 tick, between 0.005 and 0.995; never a market order
- A sell never exceeds the quantity held, so it cannot silently flip into the opposite side; short views are explicit NO buys
- Unique idempotency key; retries reuse it
- Fair value younger than the mode's maximum age (table below), with at least one external source live

- Position, market and scenario caps not exceeded. Scenario caps use the simulator once built; until then, a v0 tags each market's lean (D / R / none) and caps net exposure to a uniform polling miss either way
- Rate budget: under 25 of 30 writes and 80 of 100 reads per minute, leaving headroom for reconciliation and cancels

Limits by mode (set via `POST /mode`)

Limit	Manual mode (until Nov 2)	Election mode (Nov 3–4)
Automated strategies	None; the bot only logs	S4
Max per race	5% of balance	10%
Max per order	2,000 SUSQies	5,000
Max loss in any one scenario	10%	15%
Drawdown auto-kill	Not applicable	Off: prices swing too much on the night; system faults still trigger the kill
Fair value max age	30 seconds	5 seconds
Minimum cash held	50%	20% until 9:30pm IST Nov 4, then 0%

Kill switch: one dashboard button or an auto-trigger cancels all resting orders (`/orders/cancel-all`, retried until 200) and stops new orders. Auto-triggers: no external price for 60 seconds, five rate-limit or 5xx errors in a minute, or a position mismatch between local state and the API. Every auto-kill sends an immediate Telegram alert.

Exam period (Oct 17–26): only the logger runs on the server, with no automated trading. One Telegram message each evening confirms it is healthy; nothing needs your attention unless it reports a fault.

Tooling, ownership and automation

Each tool owns its own folders and never edits another's; they meet only at `shared/` and through pull requests that Arnav merges.

Folder	Owner	Work
`engine/`	Claude Code	API client, collectors, fair value, strategies, execution, risk
`models/`	Claude Code	Correlated simulator, vote-count model
`mock_exchange/`	Claude Code	Fake SIG API replaying recorded responses, for safe testing
`dashboard/`	Antigravity (IDE)	React UI; verified in its built-in browser
`data/` pipelines	Antigravity agents	Historical county results download and cleaning
`shared/`	Arnav approves changes	Schemas both sides depend on
`docs/`	Claude (this project)	PRD, architecture notes, post-mortem

Automation

- Claude Code hook: run `pytest` after every edit to `engine/` or `models/`
- Pre-commit: gitleaks secret scan, ruff lint, block any `.env` file
- GitHub Actions: tests on every push; `claude-code-action` reviews every pull request touching `engine/execution` or `engine/risk`
- Cross-model review: an Antigravity (Gemini) agent reviews Claude Code's order-handling pull requests
- Cloud server: systemd keeps the bot running; health check every 2 minutes; daily Telegram summary
- Exam period: background agents may work on `data/` and `models/` branches, plus draft branches for the dashboard shell and the mock exchange; nothing merges or deploys until reviewed

Security rules

- Two keys: a read-only key for one-off scripts like `sig_dump.py`; a read+trade key that exists only on the cloud server
- All keys share one 100-read, 30-write budget per minute, so once the bot is live, development and agents run against the mock exchange and recorded data, never the live API
- No AI agent ever receives the trade key or deploys to the server
- Risk limits change only by a human commit to `config/risk.yaml`
- Dashboard bound to localhost; reached through an SSH tunnel, never exposed publicly

Repo structure and conventions

One private GitHub repo, `predictions-cup`, Python 3.11 for the engine and React + Vite for the dashboard.

```
predictions-cup/
├── CLAUDE.md            # rules Claude Code reads every session
├── AGENTS.md            # same rules for Antigravity
├── config/              # risk.yaml, modes.yaml, market_map.yaml
├── shared/              # schemas.py → JSON Schema
├── engine/
│   ├── sig_client/      # REST + WebSocket client, rate limiter
│   ├── collectors/      # polymarket.py, kalshi.py, sig_books.py
│   ├── fair_value/
│   ├── strategies/      # s1_arb.py, s2_fair_value.py, s3_quote.py,
s4_election.py
│   ├── execution/
│   ├── risk/
│   └── api/             # FastAPI server for the dashboard
├── models/              # simulator/, vote_count/
├── mock_exchange/
├── dashboard/
├── data/                # pipelines only; raw data git-ignored
├── markets/             # one .md per market: Info-tab resolution text
├── tests/
└── docs/                # PRD, decisions log, post-mortem
```

Conventions

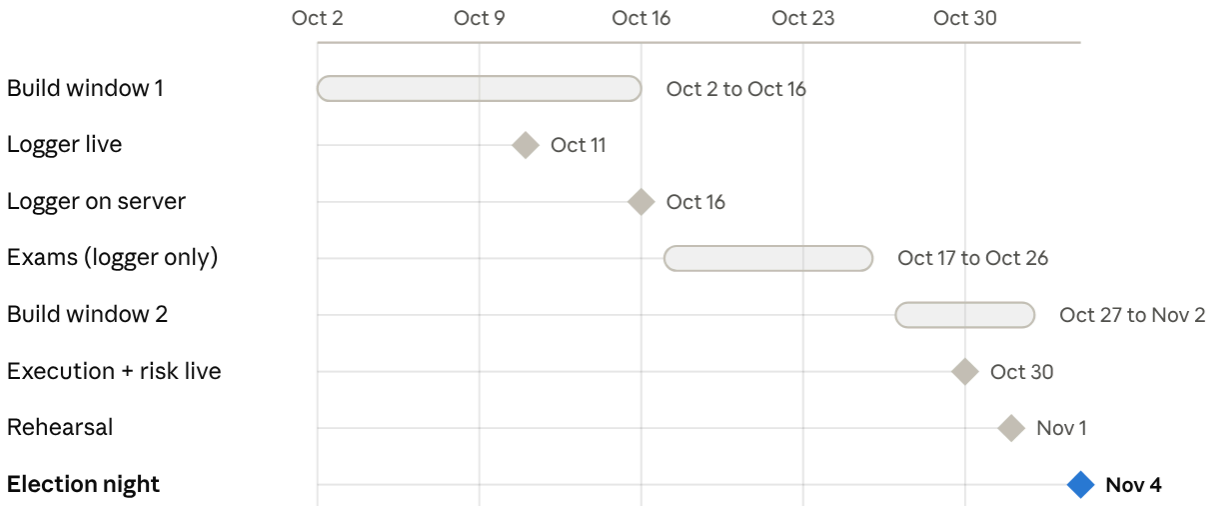
- Branch per task (`engine/execution-v1`, `dashboard/positions-panel`); pull request to `main`; Arnav merges
- Every module ships with tests against the mock exchange; no test, no merge
- All times stored in UTC; dashboard displays IST
- All prices stored YES-denominated as integer ticks (price × 200); converted to decimals only at the API boundary
- Secrets only in `.env` (git-ignored); `.env.example` committed with placeholder names
- Every trade logged with its `Signal.reason`

Timeline and milestones

The logger and manual-trading tools ship by Oct 16 and run untouched through exams; the trading system, dashboard and rehearsal fill Oct 27 – Nov 2. Tools draft branch work during exams, and nothing deploys until you review it.

The logger runs through exams; the trading system ships Oct 27 to Nov 2

Build plan by week, Oct–Nov 2026, at 1–2 hours a day



build plan · two windows around exams

Election night runs Nov 4, 4:30am to 10:30pm IST. The vote-count model does not trade live; the bot records 2026 results that night, and the model is built and evaluated on them in November.

Window 1 tasks

- [x] ~~Oct 1–2: rules read, API key, `sig_dump.py`, market analysis, trading approach decided~~
- [ ] Oct 3–5: repo, `CLAUDE.md`, `AGENTS.md`, tools installed, cloud account, SIG email sent
- [ ] Oct 6–9: SIG API client; Polymarket and Kalshi collectors; 117-race mapping (Claude Code, with Claude doing the matching)
- [ ] Oct 10–13: SQLite logger, fair value v1, `mispricings` command; manual trading starts
- [ ] Oct 14–16: logger on the server with Telegram health alerts; check it survives a restart

Exam period and Window 2 tasks

- [ ] Oct 17–26 (exams, branch work only): Antigravity drafts the dashboard shell on fake data; Claude Code drafts the mock exchange; an agent downloads Polymarket's 2024 election price history for the rehearsal
- [ ] Oct 27–28: review branch work; lag-study report; mock exchange finalised
- [ ] Oct 29–30: execution, risk manager, kill switch, chaos tests
- [ ] Oct 31: S4 strategy, dashboard on live data, results recorder
- [ ] Nov 1–2: rehearsal replaying 2024 prices; runbook; election-mode limits; deploy `v2.0`

Logging and evaluation

We log from day one because every resume number must come from real data; nothing can be reconstructed later.

Logged continuously: every quote from all three venues, every fair value with its inputs, every signal (traded or not, with reason), every order, fill, cancel, risk block and auto-kill, plus hourly balance, rank and Smart Score.

Metric	Measures	Used for
P&L: manual trades vs election night	Where profit actually came from	Post-mortem, resume
Brier score and calibration curve: our fair value vs SIG price	Whether our probabilities beat the platform's	Quant credibility
Repricing lag: time for SIG to move after Polymarket moves	Size of the speed edge	Election-night sizing, resume
Fill rate and slippage vs limit price	Execution quality	Tuning thresholds
Signals skipped by risk checks	Whether limits were too tight or loose	Post-mortem
Vote-count model (November): predicted vs final margin on recorded 2026 results	Model accuracy in real time	AI/ML resume line
Uptime and error counts	System reliability	SWE resume line

Open questions

- [x] ~~How many markets, and what kinds? 237 YES/NO markets across 117 races; no composite, multi-outcome or linked markets~~
- [x] ~~Does ALL collateral fund new fills? Not applicable: no linked markets~~
- [ ] Are there trading fees? Check the first manual fill
- [ ] How does the leaderboard value open positions before settlement?
- [ ] What happens to resting orders at the Nov 4 cutoff? (asked in the SIG email)
- [ ] Does an unscoped `/orders/cancel-all` cancel every order in one write? Test before Oct 30
- [ ] Which source does SIG resolve races on? Check the Info tabs of five markets
- [ ] How fast does SIG's market maker follow Polymarket? Answered by the lag study

- [ ] Live election-results source for the November model: free or paid; decide by Oct 28
- [ ] Which rules file does Antigravity read: `AGENTS.md` or its own workspace rules?
- [ ] Cloud server: Azure for Students in East US, with Oracle Always Free as backup
- [ ] Can we share code and aggregate results publicly after Nov 4? (asked in the SIG email)